

这是一个关于linux系统与shell脚本命令的笔记

1. linux常见命令

- ls: 显示当前路径所有文件和文件夹名称

ls -l: 也可写作ll, 作用是列出各文件和文件夹的详细信息(具体可参考<https://www.cnblogs.com/CrispyCandy/p/17586142.html>)

- pwd: 显示当前所在的绝对路径
- history: 显示历史输入命令
- cd: 进入某文件夹

cd后面不加地址, 执行时默认进入/home/用户名文件夹(当前用户身份对应的家目录)

cd -: 返回上一次所在目录

- cp: 复制文件或者文件夹

cp A B: 复制名称为A的文件, 新文件名或者路径为B

cp A ~/: 把A文件复制到家目录(家目录路径用该简便表示法)

cp -r A B: 复制名称为A的文件夹, 新文件夹名为B

cp A B C: 把A、B文件复制到C文件夹

- mv: 剪切文件或者文件夹(与cp用法相似)
- 相对路径表示法: "."表示当前路径, "../"表示上一级路径
- touch: 创建一个文本文件
- mkdir: 创建一个文件夹
- rm: 删除文件或文件夹

rm A: 删除文件A

rm -r A: 文件夹A

rm [^IKPs]*: 反向删除, 排除掉方括号中包含字母开头的文件, 其余全部删除(非常便捷)

- vim: 文本编辑

只读模式: vim A

编辑模式: i

退出编辑模式: Esc键

退出文件(不保存修改): ":q"或者":!q"

退出文件(保存修改): ":wq"

单行复制: 光标移动至目标行, 按yy复制; 光标移动至另一行, 按p粘贴至下一行

多列复制: 先ctrl+v, 移动光标, 选择区域, 按y复制; 移动光标至某一位置, 按下p后粘贴至右方

单行剪切: 光标移动至目标行, 按dd剪切; 光标移动至另一行, 按p粘贴至下一行

多列剪切：先ctrl+v，移动光标，选择区域，按d剪切；移动光标至某一位置，按下p后粘贴至右方

多行复制/剪切：按下nyy/npp后选择区域，移动光标至某一位置，按p粘贴

D/dd：整行删除

vim编辑器	R替代模式
<pre>vi file : 进入命令模式--> set nu/nonu 显示/不显示行号 /字符 查找； wq 保存并退出 i 进入编辑模式insert D或者dd 删除整行 dG删除次行至结尾 yy复制整行 5yy复制5行 p粘贴上一步记录 gg光标移动到文件首 5G光标移动到第5行 G光标移动到文件尾</pre>	<pre>:n1.n2s/word1/word2/g 替换命令</pre>

- grep：在文件中找出含某一字符串的行数

grep str file：在file中找出所有含str字符串的行

grep -An str file：在file中找出所有含str字符串的行及其后n行(after)

grep -Bn str file：在file中找出所有含str字符串的行及其前n行(before)

- more/less：浏览文件，其中more只能向下翻，less可以上下翻阅；浏览完毕时按q即可退出
- cat：将文件内容输出到屏幕(相比more/less的区别是，cat直接把文件所有内容全部输出到屏幕，而more/less则为阅读模式，仅仅显示部分内容)

cat -n A：显示A文件每行的行数

cat A B > C：将文件A、B的内容写入C中

- head：输出文件的头几行(默认10行)

head -n num file：输出file头num行

- tail：输出文件的末几行(默认10行)

tail -n num file：输出file末num行

tail -f file：实时输出最后10行

- split：把一个文件分成许多小文件

split -l n file：以n行为一组，把file分成诸多份小文件

- echo：类似于cat，也是输出文本内容

echo A >> B：把内容A写入B中

- find：找某个文件的路径

find path -name str：在路径path下的所有子目录中找到名称为str的文件的目录

- 特殊符号：有?和*两种

?: 表示单个字符, 例如: 文件xa1、xa2可写作xa?; fit、fat可写作???

: 表示任意长度的字符, 例如: 文件data、dog可写作d

- 输出重定向: >和>>(前者是完全替代原有内容, 后者是追加填入内容)
- diff: 比较两个文件是否一样(可用于检查某文件是否已经复制过)

diff A B: 比较A、B是否一样

diff -y A B: 把A、B不同的地方并列出来

- 管道命令: "|", 把管道前命令输出的内容重新当成一个文件, 使用管道后的命令进行输出

例如: head -5 file | tail -1: 该命令表示将file文件前5行当成新文件B, 之后执行输出B文件最后一行的命令(等价于输出A文件第五行)

2. shell脚本常见命令以及程序语法

- \$: 该字符有许多功能, 包括引用变量、位置参数、命令输出、表达式求值等

\$a: 表示引用变量a的值

{a}: 一样表示引用变量a的值, 相比\$a的优点在于它可以嵌在文本中

\$n: 表示引用内容的第n个参数

\$0: 输出所有引用内容

\$((a+b)): 用变量a和b的值进行代数运算, 给出结果

- awk: 逐行提取、处理数据

基本格式为: awk '{command}' file, 其中command为每行进行的操作命令, file为读取的文件; 也可用于管道命令: file | awk {command}

awk '{print \$2}' File: 打印file每一行的第二列

awk '{print NR, NF}' File: 打印每一行的行序号, 总列数(即该行的总字数), NR、NF是内置的迭代变量

awk -F,'{command}' File: 以","为分隔符(默认以空格为分割符), -F关键词用于设置分隔符

- sed: 处理、编辑文本文件

sed -e 's/str1/str2' file: '-e'表示执行后面的脚本内容, 's/'表示替换, 整个命令表示将file中的第一个str1替换成str2, 并输出到屏幕(不改变原文件)

sed -e 's/str1/str2/g' file: '/g'表示全局, 整个命令表示将file中的所有str1替换成str2, 并输出到屏幕(不改变原文件)

sed -i 's/str1/str2/g' file: '-i'表示修改原文件, 整个命令表示将file中的所有str1替换成str2

更多用法参考<https://www.runoob.com/linux/linux-comm-sed.html>

- 循环语句: 有for循环, 格式如下:

```
for i in list(e.g. list=1 2 3 4或者'seq 1 4')
do
...
done
```

- 条件判断: if语句, 格式如下:

```
if [command]; then
...
elif [command]; then
...
else
...
fi
```

- 数量关系符号: 多用于if语句

-eq: 等于
-ne: 不等于
-lt: 小于
-le: 小于等于
-gt: 大于
-ge: 大于等于

- 逻辑符号

逻辑与: &&

逻辑或: ||

字符串判断: 等于(==)、不等于(!=)

In []: